Compositional Transfer or Primitive Familiarity? What a Count-Based Prior Explains in a Typed Functional DSL

Roshan Kumar Gupta*
Department of CSE, Chandigarh University
Mohali - 140413, Punjab, India
roshankumargupta.sh@gmail.com

Abstract

Guiding program search with learned transition models is common practice in neurosymbolic synthesis. When models trained on shallow programs solve deeper, unseen tasks, this is often attributed to compositional transfer—the idea that the model learned reusable grammatical templates. We tested this assumption with a controlled ablation on a typed functional list-processing language ($N = 3,799$ programs). Standard breadth-first search (BFS) requires an average of 884.3 evaluations (median 176.5) and times out on all Depth-3 tasks under a 500-evaluation budget. However, breaking down the search engine reveals a simpler mechanism. An unguided Best-First queue with a simple length penalty accounts for 81.7% of the mean search reduction and 66.1% of the median reduction on its own (mean 208.3, median 76.0), solving 9 of 10 held-out tasks within budget and 10 of 10 when run to completion. For the remaining acceleration (151.2 evaluations on means, 51.5 on medians), a type-conditioned unigram model—which tracks single-primitive frequencies within each slot type while ignoring surrounding syntax—captures 95.6% of the remaining speedup on means (99.0% on medians) and matches the full bigram model on the hardest benchmark task (244 vs. 244 evaluations). Bigram syntax transitions add only an incremental margin of 6.7 evaluations on means and 0.5 evaluations on medians. Furthermore, on tasks requiring primitives absent from training (U_1–U_4), learned priors slow down search across all four tasks (1.3× to 3.0× slowdown, overall mean 2.6×; on Depth-3 task U_4, 1,795 vs. 589 evaluations). Because argument slots in this typed DSL belong to specific parent combinators, type checks already enforce most syntactic validity, leaving little room for transition context to help. High solve rates on transfer benchmarks can easily reflect primitive vocabulary overlap and queue mechanics rather than structural grammar learning. We provide baseline controls to help future synthesis benchmarks separate vocabulary bias from genuine compositional transfer.

Keywords: Program Synthesis, Neurosymbolic AI, Search Guidance, Compositional Generalization, Inductive Logic Programming, Empirical Methodology

1 Introduction

Inductive program synthesis searches for executable programs that satisfy a set of input-output examples [1–4]. Because candidate programs grow exponentially with expression depth, brute-force enumeration quickly becomes impractical [5,6]. Modern synthesizers tackle this exponential growth by guiding search with learned probabilistic models, prioritizing candidate syntax-tree expansions based on likelihoods learned from demonstration programs [7–9,11,12].

A central premise of learned search guidance is **compositional transfer** [9,13,14]: by learning from small, shallow programs, a model should acquire reusable structural templates—such as nesting a filter inside a map, or reducing a filtered list—and recombine them to solve deeper, unseen programs that brute-force search cannot reach within standard budgets [9,15,16].

Our study directly analyzes the mechanism underlying learned probabilistic models in syntax-guided program synthesis [22]. While foundational neurosymbolic systems condition neural recognition networks on input-output examples [7–9], they share the core hypothesis that statistical priors over syntax transitions enable scalable combinatorial search. In an initial benchmark on a typed functional list-processing language, an empirical bigram transition prior appeared to provide compelling evidence for compositional transfer, solving all 10 held-out tasks and reducing search effort from an uncensored BFS mean of 884.3 evaluations down to 57.1 evaluations.

However, observing faster search does not explain *what* causes it. Guided synthesizers differ from baseline enumeration in three distinct ways:

1. **Search Queue Mechanics:** Guided synthesizers use Best-First search with a length penalty that favors smaller programs [22]. The priority queue itself, rather than learned statistical guidance, may account for finding deep programs faster.
2. **Leaf Close Costs:** Implementation details, such as whether closing a list expression with the input variable xs incurs a length penalty, can alter queue ordering

*Roshan Kumar Gupta (roshankumargupta.sh@gmail.com) is the corresponding author.

independently of learned probabilities.

3. **Primitive Vocabulary Overlap:** Benchmarks routinely reuse the same primitive functions across training and evaluation tasks. If a model encounters `mul2` and `even` during training, simply favoring those functions will quickly solve target tasks built from the same pieces [31, 32]. The model may not have learned any grammar, but merely which primitives to try first.

1.1 Research Questions

To isolate what actually drives guided search, we investigate two core questions:

- RQ_1 **(Search Attribution):** What mechanism actually drives search acceleration in prior-guided synthesis—priority queue mechanics, single-primitive popularity, or structural transition context?
- RQ_2 **(Generalization to Unseen Primitives):** Does the learned prior support compositional transfer on tasks requiring primitives absent during training, or does it slow down search across novel-primitive tasks?

1.2 Contributions

This paper makes three main contributions:

1. **Controlled Multi-Arm Ablation Suite:** We isolate priority queue mechanics, leaf-closure penalties, unigram primitive frequencies, and structural bigram transitions under identical Best-First search conditions on a typed list-processing DSL.
2. **Empirical Attribution of Prior Guidance:** We show that unguided priority queue mechanics alone account for 81.7% of mean search reduction (66.1% on medians), while a type-conditioned unigram model captures 95.6% of the remaining guidance on means (99.0% on medians), matching the full bigram prior on the hardest benchmark task. Across all four tasks requiring unseen primitives, the learned prior degrades search performance (1.3× to 3.0× slowdown, mean 541.0 vs. 209.5 evaluations).
3. **Benchmarking Guidelines:** We outline baseline controls to help future neurosymbolic synthesis benchmarks separate primitive frequency bias from true compositional transfer.

2 Related Work

2.1 Guided Program Synthesis and Search Heuristics

Inductive program synthesis generates programs satisfying input-output specifications [1–4] through bottom-up enumeration [6], top-down deduction [5], and sketching [2]. Modern neurosymbolic systems guide discrete search using learned statistical heuristics [1, 11]. Probabilistic models over syntax trees, pioneered by Lee et al. [22] for accelerating syntax-guided synthesis, establish the core principle of guiding discrete search with syntax priors. DeepCoder [7] predicted global primitive distributions to order discrete search, while FlashFill [19] and FlashMeta [20] used deductive engines in domain-specific settings. RobustFill [8] learned end-to-end string transformations, and DreamCoder [9] coupled neural recognition models with wake-sleep library learning. Execution guidance [21] steers generation with runtime traces, while code-idiom frameworks learn recurring syntactic subtrees [15]. Related probabilistic frameworks employ learned sketches [16], property signatures (BUSTLE [18]), conditional higher-order grammars [9, 10], and branch-and-bound pruning [17, 22]. However, synthesis benchmarks frequently compare learned guidance against breadth-first enumeration, leaving the effect of priority queue mechanics unmeasured. Our study isolates queue dynamics using an explicit Uniform Search control.

2.2 Compositionality and Systematic Generalization

Machine learning systems often struggle with systematic compositionality—recombining familiar components to generalize to novel structures [23–25]. Benchmarks such as SCAN [26] and CFQ [27] demonstrate that neural sequence models frequently fail when required to generalize grammatically out of distribution. In program induction, Bayesian and neurosymbolic approaches model compositional priors over program fragments [9, 13, 14, 28–30]. We examine whether learned transition distributions capture genuine grammatical abstractions or simple vocabulary familiarity.

2.3 Shortcut Learning and Empirical Confounders

Shortcut learning occurs when models exploit superficial statistical patterns rather than true task structure [31, 32]. In natural language inference, models often rely on word overlap heuristics rather than syntactic reasoning [33, 34]. In computer vision, networks classify objects using background texture rather than shape geometry [31], while feature attribution methods show frequent reliance on spurious dataset artifacts rather than task logic [35]. In program induction, early frameworks often relied on version space algebra without isolating lexical biases [36]. We show that prior-guided synthesizers can exhibit an analogous shortcut: dramatic benchmark speedups arise simply from memorizing which primitive tokens to favor.

3 Methodology

3.1 Domain-Specific Language (DSL) Specification

We evaluate search guidance in a strongly typed functional list-processing language (List[Int] → List[Int] or List[Int] → Int) with no compound macro shortcuts. It consists of **21 atomic primitives** across four semantic categories: (1) **Combinators (4):** map : (f, xs), filter : (p, xs), fold : (init, r, xs), len : (xs); (2) **Scalar Operations (7):** id, add1, add2, add3, sub1, mul2, mul3; (3) **Predicates (7):** eq0, gt0, ge0, lt0, even, odd, mod3_0; and (4) **Reducers (3):** add (init = 0), sub (init = 0), mul (init = 1).

The terminal input list variable is xs. Program depth is defined by combinator nesting: $\text{depth}(xs) = 0$, and $\text{depth}(\text{Comb}(\text{args}, \text{child})) = 1 + \text{depth}(\text{child})$. Under the

depth bound $D \leq 3$, the space partitions into 1 program at Depth 0, 18 at Depth 1, 252 at Depth 2, and 3,528 at Depth 3, yielding a bounded search space of exactly **3,799 unique programs**. The ten calibration tasks (T_1–T_{10}) comprise: T_1 : fold$(0, \text{add}, xs)$, T_2 : len(xs), T_3 : map$(\text{mul2}, xs)$, T_4 : map$(\text{add1}, xs)$, T_5 : filter$(\text{gt0}, xs)$, T_6 : filter(even, xs), T_7 : len(filter$(\text{gt0}, xs)$), T_8 : fold$(0, \text{add}, \text{map}(\text{mul2}, xs))$, T_9 : map$(\text{add2}, xs)$, and T_{10} : filter$(\text{ge0}, xs)$, yielding 22 training transitions.

3.2 Best-First Priority Search Engine Formulation

Across all guided search conditions, candidate programs are built using Best-First Search over partial programs π with typed open holes $\mathcal{H} = [\sigma_1, \ldots, \sigma_m]$. Each slot σ is defined by a context tuple $\sigma = (\text{parent_combinator}, \text{arg_position}, \text{slot_type})$. When filling slot σ with primitive action $a \in \mathcal{A}_{\text{valid}}(\sigma)$, the partial score adds log-probabilities with a constant length penalty $\lambda = 0.05$: $\text{Score}(\pi \circ a) = \text{Score}(\pi) + \ln P(a \mid \sigma) - \lambda$. The priority queue pops prefixes minimizing priority value (Priority $= -$Score). Complete programs are evaluated against five visible input-output pairs until satisfying all tests or hitting the budget limit of $K = 500$ evaluations.

3.3 Five-Arm Causal Isolation Controls

To isolate each experimental factor, we construct five distinct search conditions:

Condition 1 (Breadth-First Baseline, BFS): Breadth-first enumeration sorted deterministically by program depth, size, category, and primitive index: $K_{\text{syn}}(P) = (\text{depth}(P), \text{size}(P), \text{cat}(P), \text{id}(P), K_{\text{syn}}(\text{child}))$. This represents standard unguided syntax search.

Condition 2 (Uniform Best-First Control): Assigns equal probability to every valid primitive in a slot: $\forall a \in \mathcal{A}_{\text{valid}}(\sigma), P_{\text{uniform}}(a \mid \sigma) = 1/|\mathcal{A}_{\text{valid}}(\sigma)|$. This keeps the Best-First queue architecture and length penalty ($\lambda = 0.05$) while removing all learned statistical bias, cleanly isolating how much the queue itself helps.

Condition 3 (Type-Conditioned Unigram Control): Tests whether search guidance requires structural syntax transitions or just single-primitive popularity. The prior is conditioned on slot type via $\mathcal{A}_{\text{valid}}(\sigma)$ but ignores parent combinators and argument positions:

$$P_{\text{unigram}}(a \mid \sigma) = \frac{c(a) + \alpha}{\sum_{a' \in \mathcal{A}_{\text{valid}}(\sigma)} (c(a') + \alpha)}$$

where $c(a)$ is the primitive count across 22 calibration transitions and $\alpha = 1.0$ is Laplace add-one smoothing. Importantly, in typed functional DSLs of this class, scalar, predicate, and reducer slots attach exclusively to their corresponding parent combinators (`ScalarOp` to `map`, `Predicate` to `filter`, `Reducer` to `fold`). Because strong typing already enforces parent-child matching by design, the type system itself silently performs most transition filtering, restricting contextual bigram choice almost entirely to combinator nesting at `ListExpr` slots.

Condition 4 (Contextual Bigram Prior, Free Close): Conditions expansion probabilities on the structural tuple $\sigma = (\text{parent}, \text{arg_pos}, \text{slot_type})$ with backoff to slot-type and uniform fallback. Closing an open list slot with xs incurs no additional score penalty ($\text{Score}_{\text{close}} = \text{Score}(\pi)$).

Condition 5 (Contextual Bigram Prior, Priced Close Control): Identical to Condition 4, but closing a list expression with xs pays the length penalty: $\text{Score}_{\text{close}} = \text{Score}(\pi) - \lambda$. We include Condition 5 as an explicit control to verify whether leaf-closure pricing confounds priority progression. As expected, Condition 5 behaves identically to Condition 4: because every valid program in this linear pipeline terminates in exactly one leaf closure (xs), penalizing closure subtracts $-\lambda$ uniformly across all completed candidates, leaving priority orderings unchanged.

3.4 Evaluation Protocol and Generalization Test

Candidate programs are checked against five visible input-output pairs during search. To evaluate practical performance under operational bounds, searches are evaluated with a search budget cap of $K = 500$ program executions. To avoid timeout distortions and measure true search effort, we also run all search conditions to completion without a budget limit across the full program space ($N = 3,799$ unique programs). In an unguided random search over $N = 3,799$ candidates, the expected rank of a unique target program is $\mathbb{E}[\text{rank}] = (N+1)/2 = 1,900$ evaluations (or bounded by $(N+1)/(m+1)$ when m candidates satisfy the visible tests). To ensure strict determinism across runs, queue items break priority ties using monotonic sequence counters.

To verify that synthesized solutions generalize correctly rather than overfitting to visible test cases, each discovered program is evaluated against a hidden suite of 20 test inputs: 9 boundary edge cases ([], [5], $[-4]$, [0], $[0, 0, 0]$, $[2, 4, -6, 8]$, $[1, 3, -5, 7]$, $[-3, 4, -5, 6, -7]$, $[2, 2, -2, -2]$) and 11 random vectors of varying lengths and ranges. A solution is certified only if it achieves 100% agreement (20/20) with reference semantics.

4 Empirical Results

4.1 Calibration Training Profile and Sample Coverage

The statistical priors were trained on 22 canonical AST transitions extracted from ten elementary calibration programs (Combinators: map 4, filter 4, fold 2, len 2; Scalars: mul2 2, add1 1, add2 1; Predicates: gt0 2, even 1, ge0 1; Reducers: add 2). In this typed DSL, these 22 transitions fully saturate permissible Depth 1–2 combinator attachments. Because type checks restrict non-combinator slots, adding training demonstrations merely re-weights primitive frequencies without adding structural degrees of freedom, explaining why unigram dominance is structurally robust.

Table 1: Synthesis search effort (evaluated candidate programs) across five search conditions on ten held-out tasks (Tasks 1–10). We report both the operational budget ($K \leq 500$) and full uncensored search to completion across the entire program space ($N = 3,799$). Bold numbers indicate the fastest condition among guided search controls. Full uncensored ranks in BFS enumeration and Uniform search are shown in parentheses for budget timeout tasks.

Task	Depth	BFS (C1)	Uniform (C2)	Unigram (C3)	Bi-Free (C4)	Bi-Priced (C5)	Hidden
T1: Filter Odds	1	15	**8**	11	11	11	20/20 (100%)
T2: Double Evens	2	115	171	**32**	**32**	**32**	20/20 (100%)
T3: Add 2 Positives	2	70	131	**30**	**30**	**30**	20/20 (100%)
T4: Count Evens	2	31	**6**	7	**6**	**6**	20/20 (100%)
T5: Count Odds	2	32	**11**	20	15	15	20/20 (100%)
T6: Sum Evens	2	241	20	**13**	19	19	20/20 (100%)
T7: Sum Positives	2	238	21	**11**	18	18	20/20 (100%)
T8: Double Pos. Evens	3	500* (1,571)	500* (955)	**244**	**244**	**244**	20/20 (100%)
T9: Sum Doubled Evens	3	500* (3,293)	412	113	**79**	**79**	20/20 (100%)
T10: Sum Add1 Evens	3	500* (3,237)	348	157	**117**	**117**	20/20 (100%)
Solved ($K \leq 500$)	—	7 / 10	9 / 10	**10 / 10**	**10 / 10**	**10 / 10**	**10 / 10**
Mean ($K \leq 500$)	—	224.2	162.8	63.8	**57.1**	**57.1**	—
Full Mean ($N = 3799$)	—	884.3	208.3	63.8	**57.1**	**57.1**	—
Full Median ($N = 3799$)	—	176.5	76.0	25.0	**24.5**	**24.5**	—

*Evaluations capped at budget $K = 500$. Values in parentheses report exact uncensored ranks in the 3,799 candidate program space.

Table 2: Synthesis effort on held-out tasks requiring primitives unseen during calibration (Tasks U_1–U_4). Lower execution counts indicate faster discovery. Across all novel-primitive tasks, unguided uniform search systematically outperforms learned statistical priors, running 2.6× faster on average and 3.0× faster on Depth-3 task U_4, showing clear negative transfer.

Task	Depth	Novel Primitives	BFS (C1)	Uniform (C2)	Unigram (C3)	Bi (C4/5)	Hidden
U_1: Triple Odds	2	mul3, odd	130	**112**	186	186	20/20 (100%)
U_2: Sub1 Evens	2	sub1	**101**	113	144	144	20/20 (100%)
U_3: Sum Mod3	2	mod3_0	243	**24**	31	39	20/20 (100%)
U_4: Trip. Sub1 Odds	3	mul3, sub1, odd	500* (1,712)	500* (**589**)	500* (1,795)	500* (1,795)	20/20 (100%)*
Mean ($K \leq 500$)	—	—	243.5	**187.2**	215.2	217.2	—
Full Mean ($N = 3799$)	—	—	546.5	**209.5**	539.0	541.0	—
Full Median ($N = 3799$)	—	—	186.5	**112.5**	165.0	165.0	—

*Budget capped at $K = 500$. Parentheses report uncensored completions. All conditions certified 20/20 on hidden test suite upon uncensored discovery.

4.2 Held-Out Benchmark Synthesis (Tasks 1–10)

All five search conditions were evaluated on the ten held-out benchmark tasks. Search effort is measured by the number of program executions required to find a satisfying program. Table 1 summarizes the results.

Canonical BFS (Condition 1) solved 7 of the 10 tasks within the 500-evaluation budget, timing out on all three Depth-3 tasks (Tasks 8, 9, and 10), which appear at syntactic ranks 1,571, 3,293, and 3,237 in the enumeration order. Across the full program space ($N = 3,799$), BFS requires an average of 884.3 evaluations (median 176.5).

Uniform Best-First Search (Condition 2) solved 9 of the 10 tasks within budget (mean 162.8 evaluations). When run to completion, Uniform search solves Task 8 at 955 evaluations, achieving an uncensored benchmark mean of 208.3 evaluations (median 76.0) without any learned prior probabilities. This unguided queue mechanism accounts for 81.7% of the mean search reduction from BFS to Bigram (884.3 → 57.1, saving 676.0 evaluations) and 66.1% of the median reduction (176.5 → 24.5, saving 100.5 evaluations). Across 50 random tie-break seeds, Uniform search achieves an uncensored mean of 227.1 ± 55.9 evaluations (median 61.9 ± 22.4), confirming that queue efficiency is structurally robust and does not depend on how primitives are ordered in code.

The Type-Conditioned Unigram Prior (Condition 3) solved all 10 tasks within budget, requiring an uncensored mean of 63.8 evaluations (median 25.0). It captures 95.6% of the remaining guidance speedup on means (208.3 → 63.8, saving 144.5 out of 151.2 evaluations) and 99.0% on medians (76.0 → 25.0, saving 51.0 out of 51.5 evaluations). On Depth-3 Task 8, Unigram matched the full Bigram Prior step-for-step, solving it in **exactly 244 evaluations**.

The Contextual Bigram Prior (Conditions 4 and 5) required an uncensored mean of 57.1 evaluations (median 24.5), adding an incremental margin of only 6.7 evaluations on means and 0.5 evaluations on medians over the unigram baseline. Both free leaf closure (Condition 4) and priced leaf closure (Condition 5) yielded identical execution counts across all ten tasks, confirming that closure penalties do

not alter search order.

4.3 Evaluation on Novel Primitives (Tasks U_1–U_4)

Synthesis benchmarks frequently claim compositional transfer by evaluating on deeper tasks while reusing familiar primitives. To separate genuine structural composition from vocabulary familiarity, we evaluate all search conditions on four tasks built from primitives absent during calibration (mul3, sub1, mod3_0, odd). If models truly learn reusable grammatical templates, structural guidance should persist across unfamiliar atomic operations. Table 2 reports the results.

Across all four novel-primitive tasks, learned statistical priors degrade search performance compared to unguided Uniform search: U_1 (186 vs. 112, 1.7× slowdown), U_2 (144 vs. 113, 1.3× slowdown), U_3 (39 vs. 24, 1.6× slowdown), and U_4 (1,795 vs. 589, 3.0× slowdown). Across the suite, unguided Uniform search achieved an uncensored mean of 209.5 evaluations (median 112.5), running 2.6× faster overall (mean 541.0 vs. 209.5; or 123.0 vs. 83.0 excluding U_4). On Depth-3 task U_4 (map(mul3, filter(odd, map(sub1, xs)))), learned priors require 1,795 evaluations—lagging even behind unguided BFS (1,712 evaluations).

To evaluate whether this degradation was caused by Laplace smoothing ($\alpha = 1.0$), we swept $\alpha \in [0.01, 10.0]$. Negative transfer persists across the entire range: for small $\alpha \leq 0.1$, the penalty sharpens (mean 893.5 to 913.5 evaluations) because novel tokens are suppressed; for large $\alpha \geq 2.0$, search effort plateaus at 538.5 evaluations—still 2.6× slower than Uniform search (209.5 evaluations). On Depth-3 task U_4, learned priors require 1,763 to 2,071 evaluations across all α (vs. 589 for Uniform).

5 Discussion and Analysis

5.1 Resolving the Research Questions

RQ_1 **(Search Attribution):** Search acceleration is driven primarily by priority queue mechanics and single-primitive popularity, not abstract grammar rules. First, the unguided Best-First priority queue with a simple length penalty ($\lambda = 0.05$) alone accounts for **81.7%** of mean search reduction and **66.1%** of median reduction from BFS to Bigram (884.3 → 208.3 mean, 176.5 → 76.0 median). It reaches Depth-3 Tasks 9 and 10 without any statistical guidance. Second, of the remaining speedup gained from learned priors (208.3 → 57.1 mean, 76.0 → 24.5 median), the type-conditioned Unigram model captures **95.6%** of the speedup on means and **99.0%** on medians, matching Bigram step-for-step on Task 8 (244 evaluations). Structural transition context adds an incremental margin of only **6.7 evaluations** on means and **0.5 evaluations** on medians. Apparent compositional transfer on held-out tasks is therefore overwhelmingly driven by queue mechanics and primitive frequency reweighting.

RQ_2 **(Generalization to Unseen Primitives):** When evaluated on tasks requiring novel primitives (U_1–U_4), the learned prior does not generalize compositionally; instead, it causes consistent slowdown across all four tasks (1.3× to 3.0× slowdown, overall mean 541.0 vs. 209.5 evaluations). On Depth-3 task U_4, learned priors incur a **3.0× slowdown** compared to Uniform search (1,795 vs. 589 evaluations) and fall behind unguided BFS (1,712 evaluations). Because the prior favors familiar tokens, it acts as a **vocabulary familiarity filter** that suppresses novel components.

5.2 Why Vocabulary Overlap and Type Systems Explain the Speedup

When test tasks reuse primitives from calibration demonstrations, common training primitives (mul2, add1, gt0, even) match target operations. Sorting the queue by unigram frequencies deprioritizes unused operations, effectively shrinking the search space. Furthermore, because typed argument slots belong to specific parent combinators, type checks already enforce syntactic validity by design. In such DSLs, learned transition models receive credit for "structural learning" when they merely re-weight primitive frequencies on top of strong type constraints.

5.3 Empirical Standards for Synthesis Benchmarks

To ensure that future neurosymbolic program synthesis benchmarks reliably distinguish compositional learning from vocabulary familiarity, we propose three practical standards:

1. **Mandatory Unigram Baselines:** Synthesis benchmarks should report a smoothed, type-conditioned unigram baseline. If single-primitive frequencies account for the vast majority (e.g., $\geq 80\%$, as a practical rule of thumb) of observed guidance speedup, benchmark gains cannot be claimed as evidence for structural grammar learning.
2. **Search Engine Parity Controls:** Guidance ablations must hold the search engine architecture constant. Benchmarking learned heuristic search against unguided breadth-first queue confuses basic queue mechanics with model intelligence.
3. **Disjoint-Vocabulary Evaluation:** To validate claims of compositional transfer, benchmarks must evaluate models on held-out tasks built from primitive subsets absent from training demonstrations. True compositional reasoning requires applying learned structural templates to unfamiliar atomic operations.

6 Threats to Validity & Limitations

Internal and Construct Validity: We removed randomness by using deterministic FIFO tie-breaking with sequence counters, guaranteeing identical search order across runs. All conditions operated over identical AST representations and test specifications. The identical performance between Condition 4 (free close) and Condition 5 (priced close) demonstrates that queue progression was unaffected by leaf-closure penalties. We tested compositional transfer

by decomposing learned guidance into uniform, unigram, and bigram variants, cleanly isolating priority queue mechanics, primitive popularity, and structural transition context.

External Validity and Limitations: Our experiments were conducted within a functional list-processing domain restricted to Depth $D \leq 3$ ($N = 3,799$ programs). By executing all search conditions uncensored to completion across the entire bounded program space (where random search takes $\mathbb{E}[\text{rank}] = 1,900$ evaluations for a unique target), we eliminated timeout distortions and measured true global search effort. While a space of 3,799 programs is small enough to permit exhaustive evaluation of the complete search space without timeouts, it is large enough to demonstrate that depth-3 chains (e.g., ranks ¿3,200) easily exceed common search budgets like $K = 500$ under naive enumeration.

Our study has specific structural limitations: (1) The DSL consists of linear pipelines without branching, binding, or lambdas. (2) The calibration corpus comprises 22 transitions from 10 tasks, and novel-primitive evaluation uses $n = 4$ tasks. (3) Because argument slots belong to specific parent combinators, type constraints enforce most syntactic validity, naturally limiting the potential advantage of bigram context over unigram frequencies. These results should therefore be interpreted as a cautionary baseline study demonstrating how easily queue mechanics and vocabulary overlap can simulate compositional transfer in benchmark settings.

7 Conclusion

In this paper, we examined what actually drives search acceleration in prior-guided program synthesis. Our experiments show that large benchmark speedups do not necessarily reflect compositional grammar learning. Instead, two simpler mechanisms explain the observed gains: priority queue mechanics with length penalties, and single-primitive frequency bias. Priority queue mechanics alone account for 81.7% of mean search reduction (66.1% on medians), while a type-conditioned unigram model accounts for 95.6% of the remaining guidance speedup on means (99.0% on medians) and matches the full bigram prior on the hardest benchmark task. Furthermore, when tasks require primitives not seen during calibration, the learned prior degrades search across all four tasks (1.3× to 3.0× slowdown, 2.6× overall mean) by actively avoiding novel tokens. When training and evaluation tasks share the same primitive vocabulary, frequency bias alone can produce search acceleration that mimics structural reasoning. Incorporating unigram baselines and out-of-vocabulary evaluation splits will help ensure that future synthesis benchmarks measure genuine compositional transfer.

References

[1] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Foundations and Trends in Programming Languages*, vol. 4, no. 1–2, pp. 1–119, 2017.

[2] A. Solar-Lezama, "Program synthesis by sketching," Ph.D. dissertation, EECS Department, University of California, Berkeley, 2008.

[3] R. Alur *et al.*, "Syntax-guided synthesis," in *Formal Methods in Computer-Aided Design (FMCAD)*, 2013, pp. 1–8.

[4] Z. Manna and R. Waldinger, "A deductive approach to program synthesis," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 2, no. 1, pp. 90–121, 1980.

[5] J. K. Feser, S. Chaudhuri, and I. Dillig, "Synthesizing data structure transformations from input-output examples," in *PLDI*, 2015, pp. 229–239.

[6] A. Udupa *et al.*, "TRANSIT: Specifying protocols with concolic snippets," in *PLDI*, 2013, pp. 287–296.

[7] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow, "DeepCoder: Learning to write programs," in *ICLR*, 2017.

[8] J. Devlin, J. Uesato, S. Bhupatiraju, R. Singh, A. R. Mohamed, and P. Kohli, "RobustFill: Neural program learning under noisy I/O," in *ICML*, 2017, pp. 990–998.

[9] K. Ellis *et al.*, "DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning," in *PLDI*, 2021, pp. 835–850.

[10] A. K. Menon, T. Tamuz, S. Gulwani, B. Lampson, and A. Kalai, "A compositional machine learning approach to code synthesis," in *ICML*, 2013, pp. 446–454.

[11] S. Chaudhuri, K. Ellis, O. Polozov, R. Singh, A. Solar-Lezama, and Y. Yue, "Neurosymbolic programming," *Foundations and Trends in Programming Languages*, vol. 7, no. 3, pp. 158–243, 2021.

[12] R. Bunel *et al.*, "Leveraging grammar and reinforcement learning for neural program synthesis," in *ICLR*, 2018.

[13] B. M. Lake, R. Salakhutdinov, and J. B. Tenenbaum, "Human-level concept learning through probabilistic program induction," *Science*, vol. 350, no. 6266, pp. 1332–1338, 2015.

[14] J. Andreas, "Measuring compositionality in representation learning," in *ICLR*, 2019.

[15] E. C. R. Shin, M. Allamanis, M. Brockschmidt, and A. Polozov, "Program synthesis and semantic parsing with learned code idioms," in *NeurIPS*, 2019, pp. 10824–10834.

[16] M. I. Nye, L. B. Hewitt, J. B. Tenenbaum, and A. Solar-Lezama, "Learning to infer program sketches," in *ICML*, 2019, pp. 4861–4870.

[17] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[18] A. Odena and C. Sutton, "Learning to represent programs with property signatures," in *ICLR*, 2020.

[19] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," in *POPL*, 2011, pp. 317–330.

[20] O. Polozov and S. Gulwani, "FlashMeta: A framework for inductive program synthesis," in *OOPSLA*, 2015, pp. 107–126.

[21] X. Chen, C. Liu, and D. Song, "Execution-guided neural program synthesis," in *ICLR*, 2019.

[22] W. Lee, K. Heo, R. Alur, and M. Naik, "Accelerating search-based program synthesis using learned probabilistic models," in *PLDI*, 2018, pp. 436–449.

[23] J. A. Fodor and Z. W. Pylyshyn, "Connectionism and cognitive architecture: A critical analysis," *Cognition*, vol. 28, no. 1–2, pp. 3–71, 1988.

[24] G. F. Marcus, *The Algebraic Mind: Integrating Connectionism and Cognitive Science*. MIT Press, 2001.

[25] D. Hupkes, V. Dankers, M. Mul, and E. Bruni, "Compositionality decomposed: How do neural networks generalise?" *Journal of Artificial Intelligence Research*, vol. 67, pp. 757–795, 2020.

[26] B. M. Lake and M. Baroni, "Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks," in *ICML*, 2018, pp. 2873–2882.

[27] D. Keysers *et al.*, "Measuring compositional generalization: A comprehensive method on realistic data," in *ICLR*, 2020.

[28] N. D. Goodman, V. K. Mansinghka, D. M. Roy, K. Bonawitz, and J. B. Tenenbaum, "Church: A language for generative models," in *UAI*, 2008, pp. 220–229.

[29] E. Dechter, J. Malmaud, R. P. Adams, and J. B. Tenenbaum, "Bootstrap learning via modular concept discovery," in *IJCAI*, 2013, pp. 1302–1309.

[30] P. Liang, M. I. Jordan, and D. Klein, "Learning programs: A hierarchical Bayesian approach," in *ICML*, 2010, pp. 639–646.

[31] R. Geirhos *et al.*, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.

[32] D. Arpit *et al.*, "A closer look at memorization in deep networks," in *ICML*, 2017, pp. 233–242.

[33] R. T. McCoy, E. Pavlick, and T. Linzen, "Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference," in *ACL*, 2019, pp. 3428–3448.

[34] T. Niven and H. Y. Kao, "Probing neural network comprehension of natural language arguments," in *ACL*, 2019, pp. 4658–4664.

[35] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *KDD*, 2016, pp. 1135–1144.

[36] T. Lau, S. A. Wolfman, P. Domingos, and D. S. Weld, "Programming by demonstration using version space algebra," *Machine Learning*, vol. 53, no. 1–2, pp. 111–156, 2003.